FindMyThing — Secure Campus Lost & Found & Item Recovery Platform

Technical Research, Feasibility Investigation, and Decision Specification

Decision horizon: freeze material design choices before October 19; implement and demonstrate by November 6.

> **Decision principle:** maximize marks per unit of implementation effort and technical risk. The recommended system is intentionally a small modular monolith, not a showcase of unnecessary infrastructure.

1. Executive Summary

Recommendation: build FindMyThing as a **React single-page frontend + FastAPI modular monolith + Azure SQL Database + Azure Blob Storage + Power BI Desktop**, deployed with the smallest topology that the university accepts. Use server-side sessions stored in the database, Argon2id password hashing, explicit student/moderator authorization, deterministic SQL matching, private Blob Storage, and a transaction for claim approval and recovery.

This architecture directly covers all frozen MVP features and all rubric items: a running full-stack application; a relational schema with more than five natural related tables; 3NF; keys, foreign keys, constraints, joins, indexes, a view, a trigger, and a transaction; Azure relational and non-relational services; Power BI analytics; HTTPS; secrets outside Git; database firewall rules; validation against SQL injection and XSS; activity logs; and a threat model.

The strongest database recommendation is **Azure SQL Database**, not because SQL Server is inherently better than PostgreSQL, but because the current Azure SQL Database free offer provides a documented free allowance of 100,000 vCore seconds, 32 GB data, and 32 GB backup storage per database per month, with auto-pause available when the allowance is reached [1]. Azure Database for PostgreSQL Flexible Server is technically suitable and supports the needed database features, but its official pricing model bills provisioned compute and storage hourly, so it should be used only if the university supplies credits or specifically requires PostgreSQL [2].

Blob Storage is the correct non-relational store because the requirement is object/file storage for photographs. Cosmos DB would add cost, schema, and code without supporting a frozen MVP requirement. The database stores report metadata and a Blob object key; the application issues short-lived read access only after authorization.

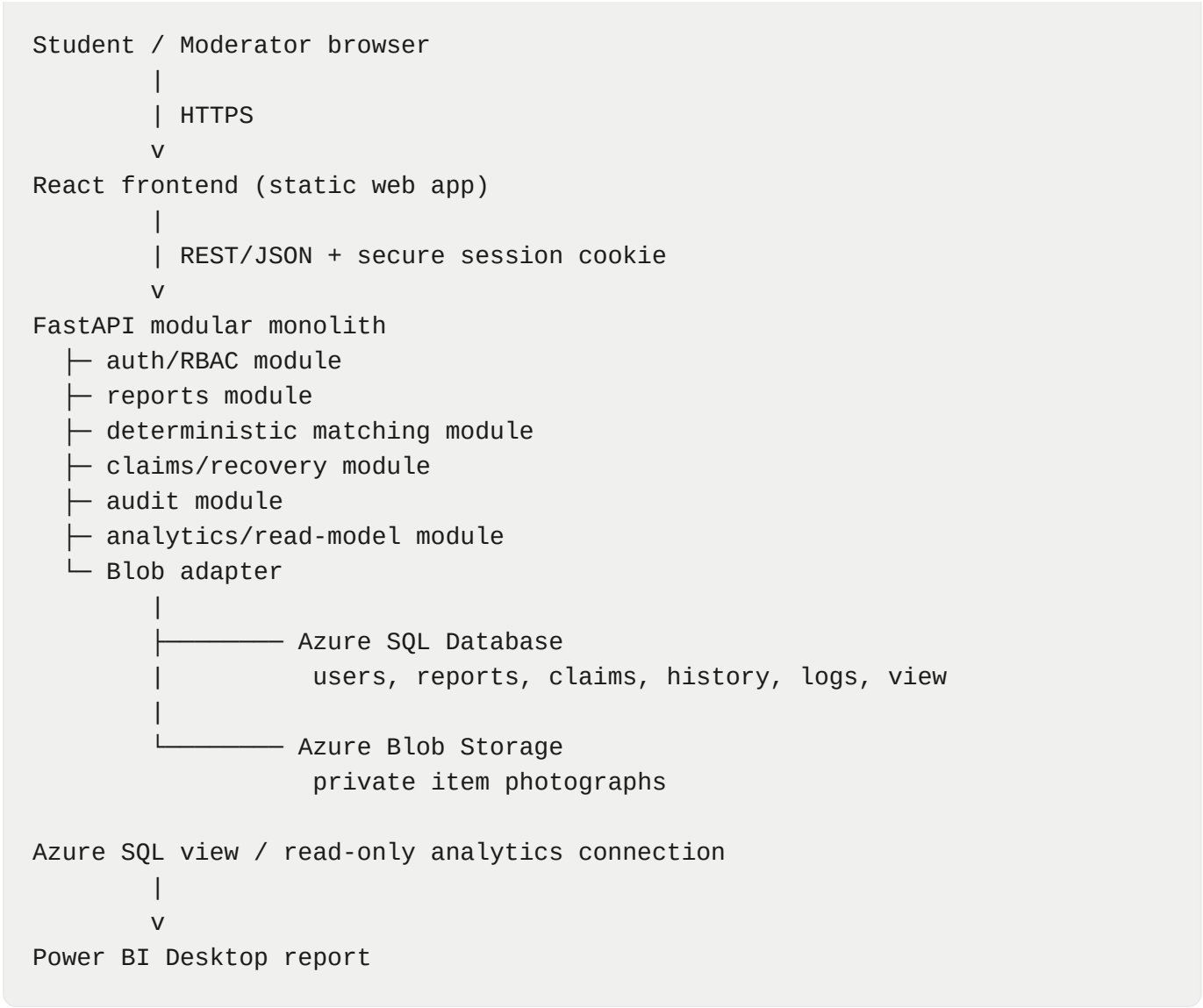
2. Rubric-to-Architecture Mapping

Rubric	Implementation evidence to demonstrate
System Design	React UI; FastAPI modules; REST endpoints; Azure SQL; Blob Storage; Power BI; data-flow and component diagrams; modular-monolith trade-off.
RDBMS	`users`, `roles`, `categories`, `locations`, `reports`, `attachments`, `matches`, `claims`, `status_history`, and `activity_logs`; PK/FK/CHECK/UNIQUE constraints; JOIN queries; indexes; `vw_recovery_analytics`; status-history trigger; claim-approval transaction.
DP-900	Structured SQL records; unstructured Blob photographs; semi-structured JSON API/audit metadata only if genuinely retained; Azure SQL relational service; Blob Storage non-relational object store; Power BI Desktop report.
Cloud Security	Login/logout; server sessions; RBAC; Argon2id; HTTPS; environment variables; parameterized queries; React safe rendering; Azure SQL firewall; audit/activity log; threat model and checklist.
Integration	React calls FastAPI; FastAPI reads/writes Azure SQL and Blob Storage; SQL view feeds Power BI; every material workflow creates an audit record.
Demo/viva	One deployable application, one database, one storage account, and simple explainable flows.

3. Recommended System Architecture

A. High-level architecture

Plain Text



B. Component responsibilities

React frontend: routes, forms, validation feedback, report browsing, match display, claim submission, moderator queues, dashboard charts, and role-aware navigation. It must not contain authorization decisions; it only improves usability.

FastAPI API: validates request data, establishes sessions, enforces RBAC, executes parameterized SQL through a small repository/data-access layer, coordinates Blob operations, calculates deterministic match scores, writes audit events, and returns DTOs that exclude passwords, hashes, internal secrets, and unnecessary personal data.

Database: remains the system of record for identity, reports, workflows, status changes, matches, claims, and audit events. The database enforces integrity with constraints rather than relying only on frontend checks. PostgreSQL documentation makes the same general point for relational design: constraints reject invalid data and include CHECK, NOT NULL, UNIQUE, primary-key, and foreign-key forms [3]. The corresponding Azure SQL features are sufficient for this schema.

Blob Storage: stores photographs as binary objects. SQL stores only the object key, media metadata, and attachment relationship. Azure Storage encrypts data at rest by default and supports HTTPS-only transfer and network controls [4].

Power BI: reads an analytics view or a small reporting query. For the time-constrained MVP, build the report in Power BI Desktop and submit the `.pbix` file or screenshots/export required by the university. Publishing to Power BI Service is optional and must not be treated as a free-tier assumption.

C. Main data flows

1. **Registration:** React sends email, display name, and password over HTTPS. API validates input, checks uniqueness, hashes with Argon2id, inserts a `STUDENT` role, and logs `REGISTRATION` without logging the password.
2. **Login:** API looks up the user by normalized email, verifies Argon2id, creates a random server-side session with expiry, sends a `Secure; HttpOnly; SameSite=Lax` cookie, and logs success or failure. Login errors should not reveal whether an email exists.
3. **Report creation:** API validates type (`LOST` / `FOUND`), category, location, timestamps, and description; inserts a report; uploads a validated photograph using a random Blob key; inserts attachment metadata; and records the event. If upload or metadata persistence fails, the API cleans up the partial object or reports a pending cleanup state.
4. **Potential matching:** when a report is created or updated, the API runs a parameterized query against opposite-type, eligible reports. A transparent score can be: category exact match 40%, location exact/near match 25%, date proximity 25%, normalized keyword overlap 10%. Store only `POTENTIAL` matches with score and explanation. Never label a match as ownership.
5. **Claim:** an authenticated student submits a claim for a `FOUND` report and identifying information. The API allows a claim only for a report not already recovered and prevents a user claiming their own found report. Sensitive identifying text is visible only to authorized moderators and the claimant.
6. **Verification/recovery:** a moderator approves or rejects. Approval runs one database transaction: lock/check claim and report, update claim, update report to `RECOVERED`, insert status history, and insert audit event. A failure rolls back all steps.
7. **Analytics:** Power BI reads `vw_recovery_analytics` or a reporting query. Operational writes remain in FastAPI; analytics does not write to production tables.

D. MVP API categories/endpoints

Authentication

- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/logout`
- `GET /api/auth/me`

Reports

- `GET /api/reports?type=&status=&category_id=&location_id=&from=&to=`
- `POST /api/reports`
- `GET /api/reports/{report_id}`
- `PATCH /api/reports/{report_id}` — owner for allowed fields; moderator for moderation fields
- `POST /api/reports/{report_id}/photo`
- `GET /api/reports/{report_id}/photo` — returns authorized short-lived access

Matching

- `GET /api/reports/{report_id}/matches`
- `POST /api/reports/{report_id}/matches/recalculate` — owner or moderator; optional if recalculation is automatic

Claims

- `POST /api/reports/{report_id}/claims`
- `GET /api/me/claims`
- `GET /api/moderation/claims?status=PENDING`
- `POST /api/moderation/claims/{claim_id}/approve`
- `POST /api/moderation/claims/{claim_id}/reject`
- `POST /api/moderation/reports/{report_id}/recover` — optional only if recovery is a separate moderator action

Analytics/audit

- `GET /api/moderation/analytics/summary`
- `GET /api/moderation/activity`

Avoid separate microservice APIs, GraphQL, WebSockets, chat, notifications, or ML endpoints.

E. Non-functional requirements

- **Security:** HTTPS; server-side authorization on every protected endpoint; Argon2id; secure cookies; CSRF protection for cookie-authenticated state changes; parameterized

SQL; output encoding; private Blob container; least-privilege secrets; audit events; file allowlist.

- **Availability:** tolerate ordinary student-scale outages; use managed Azure SQL backups; show friendly errors; no high-availability topology in MVP.
- **Performance:** normal list/filter requests target under 2 seconds in the deployed demo; paginate reports and claims; index actual filter columns; run deterministic matching server-side.
- **Maintainability:** modules by business domain; migrations; typed request/response models; `.env.example`; seed data; API documentation; tests for authorization and approval transaction.
- **Scalability:** one application instance and managed database initially; modular boundaries allow later extraction, but no microservices now.
- **Usability:** clear LOST/FOUND labels; accessible forms; responsive layout; explain "potential match" versus verified ownership; prevent duplicate submit buttons.

4. Architecture Trade-off

Primary trade-off: modular monolith versus microservices

Microservices advantages: independent deployment and scaling, stronger service ownership boundaries, and a familiar industry pattern.

Microservices disadvantages: multiple deployments, service-to-service authentication, distributed tracing, independent data ownership, more network failure modes, more secrets, more cost, and a harder viva. The MVP has one team, one deadline, modest traffic, and strongly related transactions such as claim approval plus recovery.

Modular monolith advantages: one deployable unit, one transaction boundary, one database connection strategy, simpler local development, simpler debugging, lower Azure cost, and clear internal modules. It still demonstrates separation of concerns and can be decomposed later.

Decision: use a **modular monolith**. This is not a rejection of scalability; it is a deliberate scope and risk decision. The application is modular in code and relationally disciplined while remaining operationally simple. The design review should explicitly state that a service split is a future option only after independent scaling or team-size evidence exists.

5. RDBMS Design

Use natural entities, not artificial tables made only to reach five.

Table	Purpose and important columns	PK/FK and constraints	Relationships
roles	Controlled roles: STUDENT, MODERATOR; role_id, name	PK role_id; UNIQUE name; CHECK known values if seeded	One role to many users
users	Identity and credentials: user_id, email_normalized, display_name, password_hash, role_id, is_active, timestamps	PK user_id; FK role_id; UNIQUE email; NOT NULL; CHECK length/status	User creates reports, claims, logs
categories	Controlled item categories: category_id, name	PK; UNIQUE name; NOT NULL	Category to reports
locations	Campus locations: location_id, name, optional description	PK; UNIQUE name	Location to reports
reports	Lost/found item report: report_id, report_type, title, description, category/location IDs, occurred_at, status, reporter_id, timestamps	PK; FKs to users/categories/locations; CHECK type/status; NOT NULL; status defaults ACTIVE	User/category/location to many reports
attachments	Photo metadata: attachment_id, report_id, blob_key, original_media_type, size_bytes, timestamps	PK; FK report; UNIQUE blob key; CHECK allowlisted media type and positive bounded size	One report to zero/many photos; MVP may enforce one photo in service logic
matches	Deterministic potential relation: match_id, lost report ID, found report ID, score, factors JSON/text, status, created timestamp	PK; FKs to reports; CHECK opposite type is enforced in service/trigger; UNIQUE pair; score 0–100	Report-to-report association

`claims`	Ownership claim: `claim_id`, found report ID, claimant ID, identifying information, status, reviewer, reviewed timestamp	PK; FKs; CHECK status; one pending claim per claimant/report via partial/filtered uniqueness where supported or service transaction	User claims against found report; moderator reviews
`status_history`	Immutable workflow history: `history_id`, report ID, old/new status, changed by, reason, timestamp	PK; FKs; NOT NULL	Report has many status events
`activity_logs`	Audit event: `activity_id`, actor ID nullable, event type, object type/id, result, request ID, timestamp, safe metadata	PK; FK actor nullable; CHECK event type/result; no passwords/tokens	User/action/object audit trail

Suggested status values: report `ACTIVE`, `CLOSED`, `RECOVERED`, `REJECTED`; claim `PENDING`, `APPROVED`, `REJECTED`, `WITHDRAWN`; match `POTENTIAL`, `DISMISSED`.

Important integrity constraints:

- A report must be exactly `LOST` or `FOUND`.
- A claim must target a `FOUND` report and cannot be approved if the report is already recovered.
- A moderator role is required for moderation endpoints; do not rely on a client-supplied role.
- `occurred_at` cannot be unreasonably far in the future.
- `email_normalized` is lowercase/trimmed and unique.
- Foreign keys use `ON DELETE RESTRICT` for business records; retain audit history rather than cascading away evidence.
- Store timestamps in UTC; render local time in the UI.

Representative joins

SQL

```sql
-- Browse active found reports with category, location, and reporter display na
SELECT r.report_id, r.title, c.name AS category, l.name AS location,
```

```sql
       r.occurred_at, r.status, u.display_name
FROM reports r
JOIN categories c ON c.category_id = r.category_id
JOIN locations l ON l.location_id = r.location_id
JOIN users u ON u.user_id = r.reporter_id
WHERE r.report_type = @report_type
  AND r.status = 'ACTIVE'
ORDER BY r.created_at DESC
OFFSET @offset ROWS FETCH NEXT @page_size ROWS ONLY;

-- Moderator queue with claimant and report context
SELECT cl.claim_id, cl.status, cl.created_at,
       claimant.display_name AS claimant,
       r.report_id, r.title, r.occurred_at, l.name AS location
FROM claims cl
JOIN users claimant ON claimant.user_id = cl.claimant_id
JOIN reports r ON r.report_id = cl.report_id
JOIN locations l ON l.location_id = r.location_id
WHERE cl.status = 'PENDING'
ORDER BY cl.created_at;

-- Potential matches with explanations
SELECT m.match_id, m.score, m.factors, found.report_id AS found_report_id,
       found.title AS found_title, lost.title AS lost_title
FROM matches m
JOIN reports lost ON lost.report_id = m.lost_report_id
JOIN reports found ON found.report_id = m.found_report_id
WHERE m.lost_report_id = @report_id AND m.status = 'POTENTIAL'
ORDER BY m.score DESC;
```

Indexes

Create only indexes tied to actual filters and joins. Primary keys and UNIQUE constraints already create indexes. Add:

SQL

```sql
CREATE INDEX ix_reports_browse
  ON reports (report_type, status, category_id, location_id, occurred_at DESC);
CREATE INDEX ix_reports_reporter_created
  ON reports (reporter_id, created_at DESC);
CREATE INDEX ix_claims_queue
  ON claims (status, created_at);
CREATE INDEX ix_claims_report
  ON claims (report_id, status);
CREATE INDEX ix_matches_lost_score
  ON matches (lost_report_id, status, score DESC);
```

```
CREATE INDEX ix_matches_found_score
  ON matches (found_report_id, status, score DESC);
CREATE INDEX ix_activity_time
  ON activity_logs (created_at DESC, event_type);
```

A broad description keyword search can begin with normalized token matching or a database full-text index only if needed. Do not create an index for every column; indexes speed reads but add write/storage overhead [5].

6. 3NF Analysis

1NF: each row has a key, columns contain atomic values, and repeating groups are separated. A report has one category ID and one location ID; multiple photos are rows in `attachments`, not comma-separated URLs. Multiple status changes are rows in `status_history`.

2NF: every non-key attribute depends on the whole key. The schema uses surrogate single-column keys, so partial dependency is avoided. For the composite business concept of a report pair, the unique `(lost_report_id, found_report_id)` constraint prevents duplicate match relationships.

3NF: non-key attributes depend on the key, the whole key, and nothing but the key. Category name belongs in `categories`, not repeated in reports. Location name belongs in `locations`. Role name belongs in `roles`. Reporter display name belongs in `users`. Attachment metadata belongs in `attachments`. Status-change details belong in `status_history`.

Dependency issues: `report_type` and `status` are controlled domains rather than lookup tables for the MVP; this is acceptable if CHECK constraints and application enums are consistent. `matches.factors` may be stored as a short JSON object because it is an explanatory snapshot of a calculated result, not a core relational attribute. If the university interprets “no semi-structured data” strictly, store a short textual explanation instead. Do not put mutable user/profile data into an audit snapshot unless the event genuinely requires it.

7. SQL View / Trigger or Procedure / Transaction

Analytics view

A normal SQL view is appropriate because it exposes a stable reporting shape without duplicating data. PostgreSQL documents that a regular view is not physically materialized; its query runs when referenced [6]. Azure SQL provides the equivalent logical view feature.

SQL

```sql
CREATE VIEW vw_recovery_analytics AS
SELECT
    r.report_id,
    r.report_type,
    r.status,
    c.name AS category,
    l.name AS location,
    r.created_at,
    r.occurred_at,
    MIN(CASE WHEN sh.new_status = 'RECOVERED'
             THEN sh.changed_at END) AS recovered_at,
    CASE WHEN r.status = 'RECOVERED' THEN 1 ELSE 0 END AS is_recovered,
    CASE WHEN r.status = 'RECOVERED'
         THEN DATEDIFF(day, r.created_at,
              MIN(CASE WHEN sh.new_status = 'RECOVERED'
                       THEN sh.changed_at END))
         ELSE NULL END AS days_to_recovery
FROM reports r
JOIN categories c ON c.category_id = r.category_id
JOIN locations l ON l.location_id = r.location_id
LEFT JOIN status_history sh ON sh.report_id = r.report_id
GROUP BY r.report_id, r.report_type, r.status, c.name, l.name,
         r.created_at, r.occurred_at;
```

If the implementation uses PostgreSQL rather than Azure SQL, replace `DATEDIFF` with timestamp subtraction and `EXTRACT`. The view supports total lost/found, recovered items, recovery rate, category/location distribution, trends, and average time to recovery.

Trigger choice

Use one **status-history trigger** as the legitimate database automation. It protects the audit invariant even when a report status is changed by a script or administrator. SQL Server implementation can use an `AFTER UPDATE` trigger with `inserted` and `deleted`; PostgreSQL can use an `AFTER UPDATE OF status FOR EACH ROW` trigger. PostgreSQL documents that an AFTER trigger sees the completed row changes and that row-level triggers execute once per affected row [7].

Pseudocode:

SQL

```sql
CREATE TRIGGER trg_report_status_history
AFTER UPDATE OF status ON reports
REFERENCING OLD_TABLE AS old_rows NEW_TABLE AS new_rows
FOR EACH ROW
```

```
WHEN (OLD.status <> NEW.status)
EXECUTE FUNCTION record_report_status_change();
```

The application supplies the authenticated actor through a transaction-scoped context value or explicitly writes the history row in the same transaction. The trigger should never create an unbounded recursive update.

Claim approval transaction

This is the strongest transaction for the rubric because an approval is not complete unless all related state agrees. SQL transactions provide an all-or-nothing operation; PostgreSQL describes intermediate states as invisible to other transactions until commit, with rollback cancelling all steps `8`.

SQL

```
BEGIN TRANSACTION;

-- Lock the claim and report; verify they are still eligible.
SELECT claim_id, report_id, status
FROM claims WITH (UPDLOCK, ROWLOCK)
WHERE claim_id = @claim_id AND status = 'PENDING';

SELECT report_id, status
FROM reports WITH (UPDLOCK, ROWLOCK)
WHERE report_id = @report_id AND status = 'ACTIVE';

UPDATE claims
SET status = 'APPROVED', reviewer_id = @moderator_id,
    reviewed_at = SYSUTCDATETIME()
WHERE claim_id = @claim_id AND status = 'PENDING';

UPDATE reports
SET status = 'RECOVERED', updated_at = SYSUTCDATETIME()
WHERE report_id = @report_id AND status = 'ACTIVE';

INSERT INTO status_history
(report_id, old_status, new_status, changed_by, reason, changed_at)
VALUES (@report_id, 'ACTIVE', 'RECOVERED', @moderator_id,
        'Approved ownership claim', SYSUTCDATETIME());

INSERT INTO activity_logs
(actor_id, event_type, object_type, object_id, result, created_at)
VALUES (@moderator_id, 'CLAIM_APPROVED', 'CLAIM', @claim_id,
        'SUCCESS', SYSUTCDATETIME());
```

```
COMMIT TRANSACTION;
```

If any eligibility check, update, or insert fails, issue `ROLLBACK`. The service must return a conflict rather than approving a second claim.

RDBMS deliverables

- `schema.sql`: tables, enums/CHECKs, keys, foreign keys, indexes, view, trigger/procedure.
- `queries.sql`: browse/filter JOINs, match query, moderator queue, analytics queries, and the claim-approval transaction.
- `seed.sql`: roles, categories, campus locations, two users, representative lost/found reports, one potential match, claims, and status history. Use non-production demo passwords and document them outside the committed production configuration.
- `migrations/`: numbered migration files for repeatable deployment.

8. Azure & DP-900 Architecture

Relational options

Azure SQL Database: current Microsoft documentation describes a free offer with 100,000 vCore seconds, 32 GB data, and 32 GB backup per database per month, up to 10 General Purpose databases per subscription. It can auto-pause when the free limit is reached, or continue with additional charges if explicitly selected `1`. It supports relational constraints, JOINs, views, triggers, stored procedures, transactions, firewall rules, backups, and Power BI/SQL tooling.

Azure Database for PostgreSQL Flexible Server: provides managed PostgreSQL and supports the required relational features. Its official pricing documentation says compute is billed per vCore-hour and storage/backup per GiB-month, with billing for each full hour the server exists; it is therefore not safe to describe it as free without a university credit or a currently verified offer `2`. It is a good technical choice when the student already has strong PostgreSQL skills or the course specifically expects PostgreSQL.

Decision: Azure SQL Database for the lowest cost uncertainty and easiest current free-offer story. Keep the schema portable enough that PostgreSQL is a fallback, not the primary plan.

Non-relational storage

Use **Azure Blob Storage** for photographs. Blob is object storage, not a relational table, and therefore demonstrates the DP-900 distinction without introducing a second application

database. The relational database stores `blob_key`, media type, size, and report relationship; the photograph bytes remain in Blob Storage.

Do not add Cosmos DB. A document store is not needed for fixed report/claim relationships, and Cosmos would create another security, cost, partitioning, and data-consistency topic. Blob Storage is sufficient for the stated "one non-relational store" requirement when the report clearly classifies it as unstructured object data.

Data classification

- **Structured:** users, roles, reports, categories, locations, claims, matches, status history, and activity logs in Azure SQL.
- **Unstructured:** item photographs in Azure Blob Storage.
- **Semi-structured:** not required for the MVP. If the implementation retains a small JSON `match_factors` explanation or structured request metadata, identify it honestly as semi-structured. Do not manufacture a Cosmos DB dataset solely to satisfy a classification slide.

Analytics

The simplest legitimate path is: Azure SQL → `vw_recovery_analytics` → Power BI Desktop → report pages. Include cards for total lost, total found, pending claims, recovered items, and recovery rate; charts for category/location and monthly lost/found trends; and a table of average/median time to recovery. Use a read-only database user if possible.

Power BI Desktop is the lowest-risk deliverable because it can produce a real report file without requiring a paid Power BI Service workspace. Direct publishing, scheduled refresh, sharing, gateway configuration, and organizational licensing are **not assumed free**. Verify the student/university tenant's Power BI license before promising online publication. Microsoft's connector documentation should be checked for the selected database/driver and connection mode immediately before the demo [9].

9. Azure Cost / Free-Tier Feasibility

Verified or strongly supported facts

- Azure for Students is advertised for eligible full-time university students, requires no credit card, includes a $100 credit for 12 months, and can renew annually while the user remains a student [10].
- The same page lists Azure SQL Database free monthly usage of 100,000 vCore seconds and 32 GB storage among the student free services [10].

- Azure SQL’s free-offer documentation says the allowance is lifetime per database while the offer applies, and auto-pause can prevent additional usage charges [1].
- Azure PostgreSQL Flexible Server’s pricing model charges provisioned compute and storage, and the server is billed for full hours that it exists [2].
- Blob Storage usage is metered by storage, operations, and potentially data transfer; the exact free allowance depends on the subscription, region, and current offer. Do not promise that a storage account is costless.
- Power BI Service licensing and sharing are separate from Power BI Desktop. Treat online publication as uncertain until the university confirms the license.

Minimum resources

1. One resource group.
2. One Azure SQL Database using the free offer and auto-pause behavior.
3. One Storage Account with one private Blob container.
4. One backend host and one frontend host. If App Service free/student capacity is unavailable, use the university-approved host or a single small VM/container host; do not provision multiple always-on services.
5. Power BI Desktop on the student’s computer; Service publication only if licensed.

Cost controls

- Select **auto-pause at free-limit** rather than “continue with additional charges” where the portal offers that behavior [1].
- Set usage alerts and review Cost Management after every deployment.
- Keep test photographs small and delete test blobs.
- Avoid HA, replicas, private endpoints, NAT gateways, Application Gateway, Defender add-ons, Log Analytics retention, Cosmos DB, and Kubernetes.
- Stop/delete compute and storage resources after the demo; export schema, seed data, screenshots, and the `.pbix` file first.
- Delete the resource group after grading if the university does not require the environment to remain active.

Practical decision

Proceed with Azure SQL free offer **only after the student sees the free-offer banner and zero estimated cost in their own subscription**. If the banner is absent, stop before creation and use university credit or ask the supervisor whether Azure PostgreSQL is

covered. Do not attach a payment method or select additional-charge behavior just to make the deployment work.

10. Cloud Security Architecture

Authentication decision: server session over JWT

Use a database-backed server session for this browser-only MVP. The browser receives only a random opaque session ID in a `Secure`, `HttpOnly`, `SameSite=Lax` cookie; the server stores the session hash, user ID, created time, last-used time, and expiry. This avoids putting authorization claims and long-lived tokens in browser storage. OWASP warns not to store session IDs, JWTs, or refresh tokens in `localStorage` / `sessionStorage`, and recommends protected cookies for browser sessions [11].

JWT is viable for independently deployed APIs, but it adds token expiry/refresh/revocation and browser-storage decisions that do not earn marks here. If the frontend and backend are on different origins, configure a narrow CORS allowlist and CSRF token/header protection.

Passwords

Use **Argon2id** through a maintained library. OWASP recommends Argon2id with at least 19 MiB memory, two iterations, and one parallelism degree as a minimum configuration, tuned against the target system [12]. Use a unique salt generated by the library; never store plaintext, reversible encryption, or fast SHA-256 hashes.

A practical policy is minimum 12–15 characters, maximum at least 64, allow spaces and Unicode, reject common breached passwords when practical, and do not silently truncate. OWASP's current authentication guidance says that passwords shorter than 15 are weak without MFA and that maximum length should be at least 64 to permit passphrases [13]. For a student MVP, document the policy and enforce length plus confirmation; do not create an overcomplicated password-scoring service.

Authorization

Student: register/login/logout; create reports; edit/delete only their own report while eligible; browse active reports; view their own claims; submit claims against found reports; view potential matches that the application makes available. Students cannot approve claims, change roles, read other claimants' identifying evidence, or access arbitrary Blob keys.

Moderator: review reports; hide/reject or edit moderation fields; view claims and identifying evidence; approve/reject claims; mark recovery; view activity and analytics. Moderator endpoints check the role server-side and return 403 for students.

Prevent privilege escalation by never accepting `role`, `reviewer_id`, `status=RECOVERED`, or ownership fields from a student request. Re-read the session user and authorize the target object in the service layer.

SQL injection

Use ORM/query-builder parameters or prepared statements for all values. OWASP identifies prepared statements/parameterized queries, safe stored procedures, and allow-list validation as primary defenses, and explicitly discourages relying on escaping alone [14]. Validate enums, UUIDs/integers, date ranges, sort fields, and pagination bounds. Map any dynamic sort column from a fixed server-side allowlist.

XSS

Validate length and allowed formats on input, but treat validation as a business rule rather than an XSS substitute. Render user text through React's normal escaped rendering; never use `dangerouslySetInnerHTML` for reports or claims. OWASP notes that modern frameworks reduce XSS through templating/auto-escaping but that escape hatches such as React `dangerouslySetInnerHTML` remain dangerous [15]. Add a restrictive Content Security Policy if the hosting setup permits it.

Secrets

Store database connection strings, session-signing/encryption secrets, Blob credentials, and Power BI credentials in deployment environment variables or an approved Azure secret store. Commit only `.env.example` with placeholders. Never commit passwords, access keys, SAS tokens, `.env`, production dumps, or real student identifying evidence. Key Vault is a good upgrade if already available, but a simple environment-variable deployment is lower-risk for this deadline.

HTTPS

Use the platform's HTTPS endpoint. Azure App Service supports HTTPS for the app endpoint and TLS configuration; custom-domain certificate binding may require supported tiers, so use the default platform HTTPS hostname if a custom domain would add cost [16]. Redirect HTTP to HTTPS and set secure cookies. Blob access must also use HTTPS; Microsoft recommends secure transfer and HTTPS-only SAS [4] [17].

Database network security

For Azure SQL, configure the server firewall to allow only the backend's outbound IP and the student's temporary admin IP during setup; remove the personal admin rule after testing. For PostgreSQL fallback, Azure documents that public access blocks all clients by

default until allowed IPv4 firewall rules are created, and private access removes the public endpoint [18]. Use public firewall rules for the MVP only if private networking would jeopardize the deadline; document that private endpoints are a future hardening option.

Activity logging

Log registration, login success/failure, logout, report create/update/reject, upload accepted/rejected, match calculation, claim submission, claim approval/rejection, recovery, authorization failures, and validation failures. Each event should have timestamp, actor ID if known, event type, object type/id, result, request/correlation ID, and safe reason. OWASP recommends logging authentication successes/failures, authorization failures, input-validation failures, file-upload events, and higher-risk administrative actions, while excluding unnecessary sensitive data [19]. Never log passwords, session IDs, access tokens, SAS URLs, claim secrets, or full uploaded content.

11. Threat Model

Asset	Threat	Attack vector	Mitigation
User accounts	Account takeover	Credential stuffing, weak password, stolen session	Argon2id; long passphrases; generic login errors; rate limit; secure cookies; idle/absolute expiry; logout invalidation; login audit.
Claim workflow	Unauthorized claim approval	Student calls moderator endpoint or tampers with role	Server-side RBAC; moderator-only routes; ignore client role; transaction and audit record; tests for 403.
Report data	SQL injection	Title/filter/claim text concatenated into SQL	Parameterized queries; ORM/repository; allowlisted sort fields; input bounds; least-privilege DB user.

Other users' browsers	XSS	Script in title, description, location, or claim text	React escaped rendering; no unsafe HTML; URL validation; CSP; output encoding; dependency updates.
Blob storage	Malicious upload or disclosure	Spoofed MIME, executable content, predictable URL, huge file	Allow only JPEG/PNG/WebP; signature check; size limit; random key; private container; short-lived read access; no public listing; optional scan before production.
Secrets	Secret leakage	`.env` committed, debug logs, exposed SAS/keys	Environment variables/Key Vault; secret scanning; redaction; short SAS; rotate on exposure; no secrets in Git.
Azure SQL	Unauthorized database access	Broad firewall, leaked admin credential	Narrow IP rules; TLS; least-privilege app login; separate admin login; remove temporary rules; monitor connections.
Authorization model	Privilege escalation	Mass assignment of `role`, `status`, `reviewer_id`	DTO allowlists; service-layer authorization; DB constraints; immutable audit; integration tests.
Availability	Resource exhaustion	Huge uploads, repeated login/search, Blob egress	5–10 MB image limit; pagination; rate limits; bounded filters; short SAS; cost alerts; auto-pause.

12. File Upload Security

Practical MVP policy

- Allow only `image/jpeg`, `image/png`, and `image/webp`; reject SVG, HTML, PDF, archives, scripts, and unknown formats.
- Enforce a maximum of 5 MB per photograph and one photograph per report for MVP unless the rubric requires more.
- Do not trust the browser `Content-Type`; inspect file signature/magic bytes and decode with a maintained image library. OWASP states that MIME headers can be spoofed and recommends extension allowlists, signature validation, generated filenames, size limits, authorization, and storage outside the webroot [20].
- Generate a UUID/object key such as `reports/{report_id}/{uuid}.jpg`; never use the original filename as the Blob name.
- Store the Blob container as private and disable public anonymous access. SQL stores only the object key and safe metadata.
- Enforce HTTPS/secure transfer on the storage account. Azure recommends HTTPS-only requests, minimum TLS, network controls, encryption at rest, and least-privilege SAS [4].
- On retrieval, authorize the user in FastAPI, then either stream through the backend or issue a read-only short-lived SAS for that one object. Microsoft recommends user delegation SAS where possible and least privileges/near-term expiry; a leaked SAS can be used by anyone who obtains it [17]. For a four-week student MVP, backend-mediated reads are easiest to explain; a short read-only SAS is acceptable if the SDK makes it simpler.
- Normalize/strip metadata if feasible and do not expose EXIF location. Re-encode the image server-side to a safe format if the image library supports it.
- Consider malware scanning as a documented future hardening step. The MVP is not a malware-scanning platform, but it must reject non-image formats and never execute uploaded content.
- Log accepted/rejected upload events without logging the image or SAS token.
- Protect the upload endpoint with authentication, CSRF defense for cookie sessions, rate limits, and request size limits.

13. Tech Stack Comparison

Criterion	React + FastAPI + PostgreSQL	React + Express + PostgreSQL	Simpler ASP.NET React/Az
Development speed	High if Python is known; Pydantic validation and	High if JavaScript is already strong; more choices	High if C# SQL are k

	OpenAPI help	require discipline	high lear
Learning burden	Python async, dependency injection, DB driver, migrations	JavaScript backend plus middleware/security choices	C#/.NET Azure ide substant
Security	Strong libraries; explicit session/RBAC design needed	Strong ecosystem but many middleware/configuration choices	Excellent more pla concepts
Azure deployment	Works, but Python hosting/runtime setup must be tested early	Works on Node hosting; straightforward if available	Very Azu SQL inte
Database integration	Excellent PostgreSQL; Azure SQL driver is available but less native	Excellent PostgreSQL; Azure SQL also available	Best Azu and store tooling
Maintainability	Typed API models and routers make a clear modular monolith	Easy to start; can become untyped middleware sprawl	Strong c more cer
Viva explainability	Very good: Python routes, repositories, Pydantic models	Very good if student is strong in JS	Good on
Four-week risk	Low if Python is known; Azure SQL compatibility must be tested	Low if JavaScript is known; same DB choice issue	Low if .N studied;
Community/docs	Excellent	Excellent	Excellent

Recommendation: choose the stack based on existing student competence. The default decision for this brief is **React + FastAPI + Azure SQL Database**, because FastAPI gives explicit validation/OpenAPI and a compact Python codebase, while Azure SQL is the lowest-cost Azure database choice. If the student has materially stronger JavaScript/Express experience, Stack B is an acceptable fallback; do not switch frameworks merely for fashion. If the course explicitly requires PostgreSQL, use Azure PostgreSQL only with verified credit.

14. Recommended Final Tech Stack

- **Frontend:** React + TypeScript + Vite; React Router; a small component library or plain accessible CSS.
- **Backend:** FastAPI + Python 3.11/3.12; Pydantic models; Uvicorn/Gunicorn or the hosting platform’ s supported command.

- **Database access:** SQLAlchemy Core/ORM or a thin parameterized repository; Alembic migrations; Azure SQL driver (`pyodbc` /Microsoft-supported driver) selected and tested in week 1.
- **Database:** Azure SQL Database free offer with auto-pause, subject to portal verification.
- **File storage:** Azure Blob Storage, private container.
- **Authentication:** database-backed opaque sessions, Argon2id, secure cookie, CSRF token for state-changing requests.
- **Analytics:** SQL view + Power BI Desktop; Service only if the student' s tenant confirms licensing.
- **Testing:** Pytest for services/security; API tests for student/moderator permissions; SQL migration test; one transaction rollback test; upload validation tests.
- **Excluded:** LLM matching, vector search, vision, chat, notifications, mobile app, microservices, Kubernetes, Kafka, Redis, payments.

15. Deployment Architecture

Smallest acceptable topology

1. **Frontend hosting:** static React build on the university-approved static host or Azure Static Web Apps if available under student/free resources.
2. **Backend hosting:** one Azure App Service or equivalent university-provided web host running FastAPI. If App Service pricing is not covered, use one small approved VM/container host rather than adding several services.
3. **Database:** one Azure SQL Database free-offer database; firewall limited to backend outbound IP and temporary administrator IP.
4. **Blob:** one storage account, private container, HTTPS-only, small test objects, minimal access key/SAS permissions.
5. **Configuration:** environment variables; optional Key Vault if already provisioned at no additional cost.
6. **HTTPS:** platform hostname and TLS; no paid custom domain required.
7. **Analytics:** Power BI Desktop from a read-only SQL user or a copied reporting database only if necessary.

Keep frontend and backend on the same site/origin if possible to simplify cookies and CSRF. If separate origins are required, configure exact allowed origins, `Allow-Credentials` , secure cookie domain/path, and CSRF header validation. Use deployment slots, private endpoints, WAF, or CI/CD promotion only if already supplied; they are not MVP requirements.

16. Earlier Subject Mapping

The student must verify the actual curriculum before submitting. Do **not** claim completion of any subject without the transcript/module list.

- **Programming:** supports Python/JavaScript syntax, functions, modules, errors, and testing.
- **Object-Oriented Programming:** supports domain models, service/repository separation, and maintainable code.
- **Web Development:** supports React, HTTP, REST, forms, cookies, validation, and browser security.
- **Database Management/RDBMS:** supports ER modelling, SQL, normalization, constraints, joins, indexes, views, triggers, and transactions.
- **Computer Networks:** supports HTTPS/TLS, DNS, ports, CORS, firewall rules, and client/server flow.
- **Software Engineering/System Design:** supports requirements, architecture diagrams, trade-offs, testing, version control, and project planning.
- **Data Fundamentals/Cloud Computing:** supports Azure resources, service models, Blob Storage, relational versus non-relational data, and DP-900 concepts.

Use only the subjects that appear in the student’ s real course history.

17. Registration Form — Copy/Paste Answers

A. System Design

FindMyThing will use a secure modular-monolith architecture with a React frontend, FastAPI REST API, Azure SQL Database, and Azure Blob Storage. Students report lost/found items, discover deterministic potential matches, submit claims, and track recovery, while moderators review reports and verify claims. The design prioritizes simple deployment, clear module boundaries, and explainability over microservices.

B. RDBMS

The system uses a normalized relational model for users, roles, reports, categories, locations, attachments, matches, claims, status history, and activity logs. It demonstrates primary/foreign keys, constraints, joins, indexes, an analytics view, a status-history trigger, and a transaction that atomically approves a claim and marks a report recovered.

C. DP-900

Structured application and workflow data will be stored in Azure SQL Database, while unstructured item photographs will be stored in private Azure Blob Storage. A SQL analytics view will feed a Power BI Desktop report showing recovery rate, trends, categories, locations, pending claims, and average time to recovery; no unnecessary Cosmos DB store is proposed.

D. Cloud Security

The application will use HTTPS, Argon2id password hashing, secure server-side sessions, role-based authorization for students and moderators, parameterized SQL, React-safe output rendering, environment-based secrets, Azure SQL firewall rules, private Blob Storage with short-lived authorized access, file-upload validation, and an activity log. A threat model covers account takeover, privilege escalation, unauthorized approval, injection, XSS, malicious files, secret leakage, and database exposure.

E. Earlier subjects

Programming, Object-Oriented Programming, Web Development, Database Management/RDBMS, Computer Networks, Software Engineering/System Design, and Cloud/Data Fundamentals are logically relevant; the student must verify which of these subjects were actually completed before submission.

F. Planned tech stack

React + TypeScript frontend; FastAPI + Python backend; Azure SQL Database; Azure Blob Storage; server-side sessions with Argon2id; REST/JSON APIs; SQL analytics view; Power BI Desktop; deployed with the smallest university/Azure-supported HTTPS topology.

18. Design Review Checklist

Prepare:

- frozen functional requirements for F1–F6;
- non-functional requirements with measurable demo targets;
- component and deployment architecture diagram;
- authentication and authorization flow diagram;
- report creation/upload/matching/claim/recovery data-flow diagram;
- ER diagram showing all natural entities and cardinalities;
- 3NF explanation with one example of each normal form;

- `schema.sql`, migrations, indexes, view, trigger, and seed data;
- representative JOIN queries and the claim-approval transaction;
- API list with request/response examples and role matrix;
- threat model and security checklist;
- Azure DP-900 classification slide;
- Azure resource diagram and firewall/HTTPS/secrets evidence;
- Power BI screenshots or `.pbix` report;
- test evidence for registration/login, 403 authorization, SQL injection payload handling, XSS rendering, upload rejection, and rollback;
- one written architecture trade-off;
- backup/export and shutdown plan.

Decisions to freeze before October 19

1. Azure SQL versus Azure PostgreSQL; default: Azure SQL after verifying the free-offer banner.
2. React + FastAPI versus Express based on the student's existing competence; default: FastAPI.
3. Session-cookie design, CSRF approach, and password library.
4. Exact report/claim/status values and the approval transaction.
5. Blob container privacy, allowed image types, size limit, and retrieval method.
6. Hosting locations and backend outbound IP needed for the database firewall.
7. Power BI Desktop deliverable and whether the university tenant permits Service publication.
8. Team ownership and the cut-off for excluded features.

19. Risks / Unknowns

1. **Azure SQL free offer/subscription compatibility:** Microsoft documents the free offer, but the portal may show different eligibility/behavior by subscription type. Verify the zero-cost banner and auto-pause setting in the student's actual subscription before provisioning `1`.
2. **Azure for Students eligibility:** requires the student to meet Microsoft's current eligibility rules and renewals; confirm institutional email and subscription state `10`.

3. **Backend hosting cost:** Azure SQL and student credits do not automatically make every App Service, Static Web Apps tier, logging feature, or network component free. Confirm the exact SKU and region before deployment.
4. **Power BI licensing:** Desktop is the safe deliverable; Service sharing/refresh may require a paid or university-provided license. Confirm with the university tenant before promising a public online report.
5. **Azure SQL driver compatibility:** test FastAPI connection, migrations, UUID handling, date/time behavior, and trigger syntax in week 1. If it creates substantial friction, switch to PostgreSQL only with verified credit.
6. **Cross-origin cookies:** avoid separate origins initially; if unavoidable, test CORS, CSRF, and secure-cookie behavior in the deployed HTTPS environment rather than localhost alone.
7. **Image decoding and scanning:** full malware scanning is out of scope; document defense-in-depth validation and the limitation honestly.
8. **Privacy/retention:** confirm whether the university requires deletion or retention of photographs, claimant evidence, and audit logs. Minimize data and provide moderator-only access.
9. **Date and rubric interpretation:** the brief states November 6 as the implementation target and October 19 as the design-freeze checkpoint; confirm both dates and whether the faculty accepts Power BI Desktop screenshots as the analytics view.

20. Final Decision Table

Decision	Options Considered	Recommended	Confidence	Reason
Frontend	React; server-rendered UI; Angular	React + TypeScript	High	Fits frozen browser MVP, strong ecosystem, easy role-aware UI.
Backend	FastAPI; Express; ASP.NET Core	FastAPI modular monolith	Medium-High	Compact, explicit validation/OpenAPI, good viva story; switch to Express only if student is substantially stronger in JS.

Database	Azure PostgreSQL Flexible Server; Azure SQL Database	Azure SQL Database free offer	High, subject to portal check	Current free offer is documented; required relational features and firewall/Power BI fit.
File storage	Azure Blob Storage; Cosmos DB; DB BLOB column	Private Azure Blob Storage	High	Correct object store for photographs; satisfies non-relational requirement without extra database complexity.
Analytics	Power BI Desktop; Power BI Service; custom dashboard only	Power BI Desktop over SQL view	High	Real analytics artifact with no assumption of paid Service licensing.
Authentication	Server sessions; JWT; external identity provider	Database-backed opaque sessions	High	Lowest browser/security complexity for one web app; secure cookie avoids browser token storage.
Password hashing	Argon2id; bcrypt; PBKDF2	Argon2id	High	OWASP’s preferred modern password KDF; tune and benchmark on target host.
Deployment	Microservices; Kubernetes; multiple PaaS services; small monolith	One frontend, one backend, one SQL DB, one Blob account	High	Smallest topology that satisfies integration, Azure, security, and viva requirements.

Architecture	Modular monolith; microservices	Modular monolith	High	Lower implementation and operational risk; still demonstrates clear internal modules and future extraction boundaries.

References

[1] Deploy Azure SQL Database for free
[2] Azure Database for PostgreSQL pricing
[3] PostgreSQL constraints
[4] Security recommendations for Blob Storage
[5] PostgreSQL indexes
[6] PostgreSQL CREATE VIEW
[7] PostgreSQL CREATE TRIGGER
[8] PostgreSQL transactions
[9] Power Query PostgreSQL database connector
[10] Azure for Students
[11] OWASP Session Management Cheat Sheet
[12] OWASP Password Storage Cheat Sheet
[13] OWASP Authentication Cheat Sheet
[14] OWASP SQL Injection Prevention Cheat Sheet
[15] OWASP Cross Site Scripting Prevention Cheat Sheet
[16] Enable HTTPS for a custom domain in Azure App Service
[17] Grant limited access to Azure Storage resources using SAS
[18] Firewall rules in Azure Database for PostgreSQL Flexible Server
[19] OWASP Logging Cheat Sheet
[20] OWASP File Upload Cheat Sheet